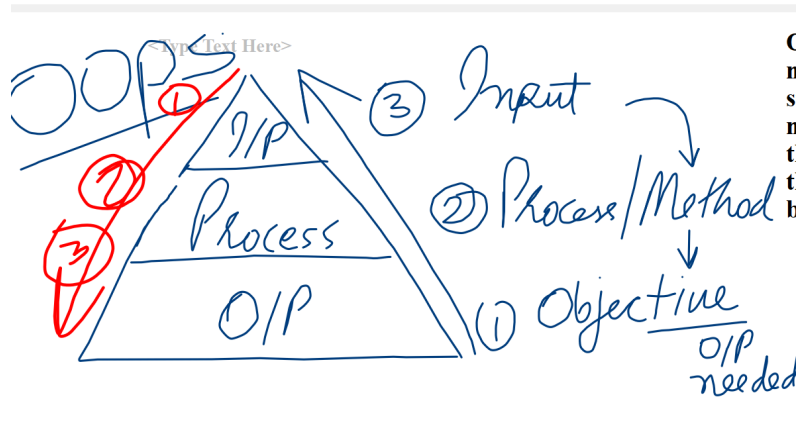
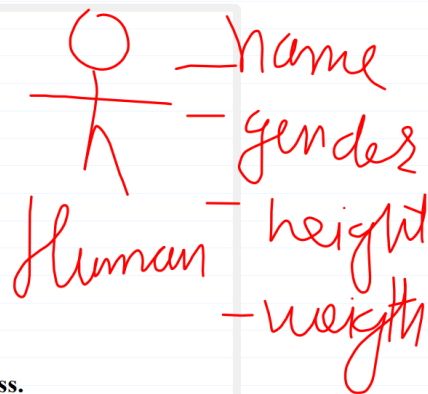




Object-oriented programming is a natural form of thinking where we start thinking from the objective needed, the process which will get us that objective, and the input needed for the process to get the objective. It's a bottom-to-top approach to thinking.

class
 Class is a virtual idea.
 A class provides a framework.
 A class will provide a set of properties.
 A class will have a set of operations to access set properties.
 ex: in intro you will say name, native, education

Object
 Object is a physical entity categorized by he/she/it.
 Object is the implementation of a framework provided by a class.
 An object will have values for the properties given by the class.
 An object will use methods to set, access, and delete properties and values.



```
class <Name>
{
    access specifier://private
        data members
    access specifier://public
        function members
};
```

In C++ we have three types of access specifiers. Depending on need one can choose.

1. public:

Can be accessible by anyone from anywhere

2. private:

Can be accessed only by class members within the class itself. Highest security because it cannot even be inherited or accessed outside the class

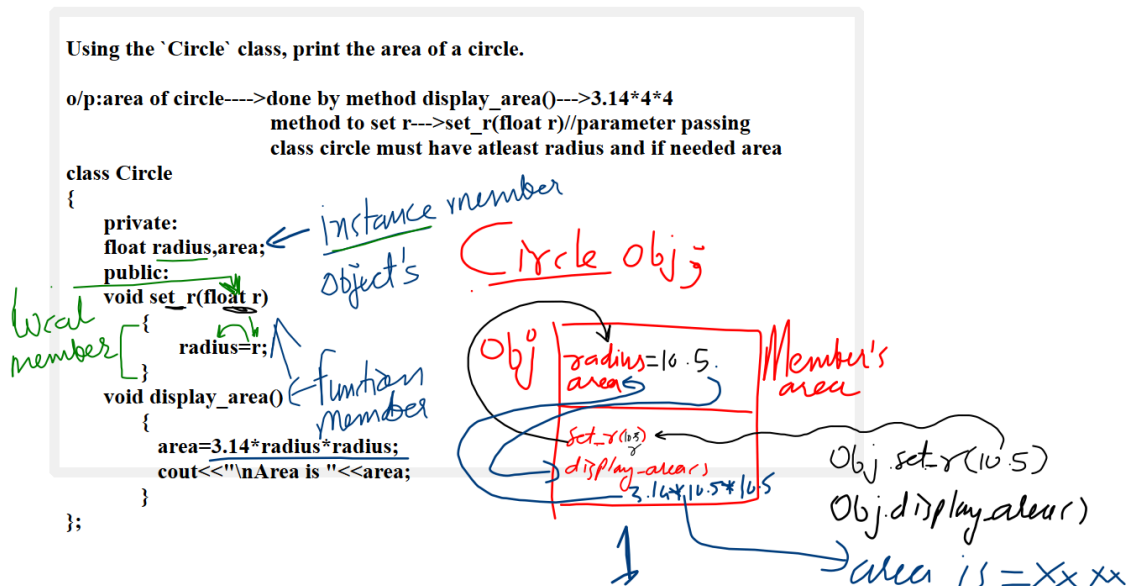
3. protected:

It is similar to 'private'. That means only members can access it, and access is within the class only but the difference is that 'protected' can be inherited.

example: gender(public), password(private), protected(family wealth)

In class design we keep data private so that outsiders cannot directly access it.

We also provide public methods to access private data via controlled actions. This is because methods are written by the developer. He or she can decide what to give, how much, and how.



```

#include <iostream>
using namespace std;
/*
  
```

Constructor:

It is a function with the exact identifier of the class itself has no return type.
 Called on its own whenever an object is created

use for one of the three reasons:

First, giving initial value :Default constructor

Giving user-supplied parameterized value :Parameterize constructor.

Making a copy of a pre-existing object :Copy constructor

A constructor is a special member function used for **initializing an object**.

Rules:

1. Same name as class
2. No return type
3. Automatically called when object is created

```

*/
  
```

```

class Human
{
  
```

```

private:
  
```

```

    string name;
    int age;
    string gender;
  
```

```

public:
  
```

```

    Human()//First, giving initial value :Default constructor
  
```

```

    {
        cout<<"\nDefault constructor call ";
        name="un-named";
        age=0;
        gender="unknown";
    }

// Method to set all details
Human(string name, int age, string gender)
//Giving user-supplied parameterized value :Parameterize constructor.
{
    cout<<"\nParameterized constructor call ";
    this->name = name;
    this->age = age;
    this->gender = gender;
}
Human(const Human &h)
//Giving user-supplied parameterized value :Parameterize constructor.
{
    cout<<"\nCopy constructor call ";
    this->name = h.name;
    this->age = h.age;
    this->gender = h.gender;
}

// Method to display details
void display_detail()
{
    cout << "\nName   : " << name << endl;
    cout << "Age    : " << age << endl;
    cout << "Gender : " << gender << endl;
}

// Method to check voting eligibility
void can_vote()
{
    if (age >= 18)
    {
        cout << "Yes, you can vote." << endl;
    }
    else
    {
        cout << "You cannot vote." << endl;
    }
}

```

```
};

int main()
{
    Human h1;
    h1.display_detail();//default
    Human h2("wonder woman",401,"female");//parameterized
    h2.display_detail();
    Human h3(h2);//copy constructor
    h3.display_detail();

    return 0;
}
```

Constructor	Normal Function
Same name as class	Any valid name
No return type	Has return type
Automatically called	Usually explicitly called
Initializes object	Performs operation
Called during object creation	Called when required

Destructor:

Rules:

- Same name as class
- Starts with ~
- No return type
- Takes no parameters
- Only one destructor per class

```
#include <iostream>
using namespace std;
/*
```

Constructor:

It is a function with the exact identifier of the class itself has no return type.
Called on its own whenever an object is created

use for one of the three reasons:

First, giving initial value :Default constructor

Giving user-supplied parameterized value :Parameterize constructor.

Making a copy of a pre-existing object :Copy constructor */

```
class Human
{
private:
    string name;
    int age;
    string gender;

public:
    Human(string name, int age, string gender)

    {
        cout<<"\nParameterized constructor call ";
        this->name = name;
        this->age = age;
        this->gender = gender;
    }
    void set_details(string name, int age, string gender)

    {
        cout<<"\nNew detail has been set. ";
        this->name = name;
        this->age = age;
        this->gender = gender;
    }
    ~Human()
    {
        cout<<"R.I.P:....."<<name;
    }

    // Method to display details
    void display_detail()
    {
        cout << "\nName   : " << name << endl;
        cout << "Age     : " << age << endl;
        cout << "Gender : " << gender << endl;
    }

};

int main()
{
    Human h("wonder woman",401,"female");//parameterized
    h.display_detail();
}
```

```

        h.set_details("super man",30,"male");
        h.display_detail();
        return 0;
    }

```

```

#include <iostream>
using namespace std;
/*

```

The lifespan of an object depends on where it is declared.

If an object is declared in a local span,

it would be removed as soon as the local span is over.

Scope of object :A scope of an object is limited to its local control area.

```
*/

```

```
class Human

```

```
{

```

```
private:

```

```
    string name;

```

```
public:

```

```
    Human(string name)

```

```
{

```

```
    this->name = name;

```

```
    cout<<"\nA human called "<<this->name<<" is created.";

```

```
}

```

```
    ~Human()

```

```
{

```

```
    cout<<"\nR.I.P:....."<<name;

```

```
}

```

```
};

```

```
int main()

```

```
{

```

```
    Human h1("Shaktiman");

```

```
    Human h2("deadpool");

```

```
    Human h3("Hulk");

```

//In this we will use a stack memory so first Shaktiman goes in, then Deadpool, and then Hulk.

//At the time of destructor call, first the topmost is removed (that is, Hulk), followed by Deadpool, followed by Shaktiman.

```
    return 0;

```

```
}

```

```

#include <iostream>
using namespace std;
/*
The lifespan of an object depends on where it is declared.
If an object is declared in a local span,
it would be removed as soon as the local span is over.
Scope of object :A scope of an object is limited to its local control area.
*/
class Human
{
private:
    string name;
public:
    Human(string name)

    {
        this->name = name;
        cout<<"\nA human called "<<this->name<<" is created.";
    }

    ~Human()
    {
        cout<<"\nR.I.P:....."<<name;
    }
};

int main()
{
    Human h1("Shaktiman");
    {
        Human h2("deadpool");
    }
    Human h3("Hulk");
//In this:
//1. H1 is created and goes on the stack.
//2. H2 is created and goes on the stack but the moment control scope is over, H2 is
removed from memory.
//3. H3 goes in and at the end of the program H3 is removed, followed by H1.
    return 0;
}

```

```

#include <iostream>
using namespace std;
/*
Handling of object with pointer

```

```

*/
class Human
{
private:
    string name;
public:
    Human(string name)

    {
        this->name = name;
        cout<<"\nA human called "<<this->name<<" is created.";
    }

    ~Human()
    {
        cout<<"\nR.I.P:....."<<name;
    }
};

int main()
{
    Human *p=new Human("Shaktiman");
    cout<<"\n somethings....";
    cout<<"\n somethings....";
    cout<<"\n somethings....";
    cout<<"\n somethings....";
    cout<<"\n somethings....";
    delete p;
    //With dynamic memory constraint if we are using a pointer to allocate memory, we
    will need to call `delete`. Otherwise memory will not get freed.
    //Human h("super");

    return 0;
}

```

```

#include <iostream>
using namespace std;
/*

```

friend function:

It is a special type of function which is declared within a class defined outside the class.

We use `friend` as a keyword before its prototype.

It is defined independently outside the class.

Access private, protected, and public data of a class like a member.

One should remember that to a friend you will send an object and then it can access. Direct access not possible

A friend can be a friend of multiple classes, allowing inter-class calculation and communication.

```
*/
class Human
{
private:
    string name;
public:
    Human(string name)

    {
        this->name = name;
        cout<<"\nA human called "<<this->name<<" is created.";
    }

    friend void secrate_hai(Human h); //Current function is declared as a friend to the
current class.
};

void secrate_hai(Human h) //Definition of friend function
{
    cout<<"\n Done by friend of Human: object has secrate name:"<<h.name;
}
int main()
{
    Human h("tatyawinchu");
    secrate_hai(h); //A friend function is called independently without any object because
it is not part of anyone.
    return 0;
}
```

```
#include <iostream>
using namespace std;
```

```
class B; // Forward declaration
```

```
class A
{
private:
    int x;
```

```
public:
```

```

    A()
    {
        x = 10;
    }

    friend void display(A o_a, B o_b);
};

class B
{
    private:
        int y;

    public:
        B()
        {
            y = 20;
        }

        friend void display(A o_a, B o_b);
};

void display(A o_a, B o_b)
{
    cout << "A x = " << o_a.x << endl;
    cout << "B y = " << o_b.y << endl;
}

int main()
{
    A a;
    B b;

    display(a, b);

    return 0;
}

```

Operator Overloading in C++ — Theory Notes

1. Definition

Operator overloading is a feature of C++ that allows us to define or redefine the behavior of an **existing operator for user-defined data types**, such as classes and objects.

For example:

```
c3 = c1 + c2;
```

If `c1` and `c2` are objects, C++ does not automatically know how to add them. We can define the behavior of `+` using **operator overloading**.

2. Why Operator Overloading?

Operators such as:

```
+ - * / == < > ++ --
```

are already defined for built-in data types.

For example:

```
a + b
```

works when `a` and `b` are integers.

But when we create our own class:

```
Complex c1, c2;
```

we may want:

```
c1 + c2
```

to perform complex-number addition.

Operator overloading allows objects to be used in a **simple, natural and readable way**.

3. Where Is It Used?

Operator overloading is useful when objects represent values for which an operator has a natural meaning.

Common examples:

- **Complex numbers** → `c1 + c2`
 - **Matrices** → `m1 + m2`
 - **Vectors** → `v1 + v2`
 - **Fractions** → `f1 + f2`
 - **Time** → `t1 + t2`
 - **Money** → `m1 + m2`
 - **Comparing objects** → `s1 > s2`
 - **Displaying objects** → `cout << obj`
-

4. How Does It Work?

In C++, an operator is treated as a special function.

For example:

`a + b`

can be implemented using:

`operator+`

Similarly:

<code>+</code>	→ <code>operator+</code>
<code>-</code>	→ <code>operator-</code>
<code>*</code>	→ <code>operator*</code>
<code>==</code>	→ <code>operator==</code>
<code><</code>	→ <code>operator<</code>
<code>[]</code>	→ <code>operator[]</code>
<code>()</code>	→ <code>operator()</code>

The keyword used is:

`operator`

followed by the operator symbol.

```
#include <iostream>
using namespace std;
```

```

class Complex
{
private:
    int img,rel;

public:
    void set_amount(int rel,int img)
    {
        this->rel=rel;
        this->img=img;
    }
    void display()
    {
        cout<<"\n"<<rel<<" + "<<img<<" i";
    }
    friend Complex operator+(Complex n1,Complex n2);
};
Complex operator+(Complex n1,Complex n2)
{
    Complex ans;
    ans.rel=n1.rel+n2.rel;
    ans.img=n1.img+n2.img;
    return ans;//We are returning an answer, which will be stored in the `main` part and
then can be used to display.
}
int main()
{
    Complex c1;
    c1.set_amount(5,3);
    Complex c2;
    c2.set_amount(3,7);
    c1.display();
    c2.display();
    Complex total=c1+c2;
    total.display();
    return 0;
}

```

Operators You Cannot Overload

Some operators **cannot** be overloaded:

::
.
.*
?:
sizeof

```
int a=10;  
int b=20;  
int c=a+b; //Here `int` is a primitive data type and `+` is designed to operate with it.
```

complex number--->created class-->user defined data type

Complex c1--->3 + 5j
 Real Imaginary

Complex c2---->2 + 3j

Complex c3=c1+c2;

//Problem is, the `+` operator is not designed to work with complex numbers as complex numbers are user-defined data not primitive data.

If we still want to use `+` on custom data, you have to use operator overloading.

For overloading an operator we have a special function, which can be a member function or a friend function.

syntax:

```
return type operator(symbol)(parameters)  
{
```

```
}
```

```
Complex operator+(Complex n1,Complex n2)
```

```
{
```

```
    Complex ans; //local
```

```
    ans.real=n1.real+n2.real;
```

```
    ans.imaginary=n1.imaginary+n2.imaginary;
```

```
    return(ans);
```

```
}
```

n1

n2

n3=n1+n2; //internally: operator+(n1,n2)

```
#include <iostream>  
using namespace std;
```

```
class Complex
```

```
{
```

```
private:
```

```
    int img,rel;
```

```
public:
```

```
    void set_amount(int rel,int img)
```

```
{
```

```

        this->rel=rel;
        this->img=img;
    }
    void display()
    {
        cout<<"\n"<<rel<<" + "<<img<<" i";
    }
    friend Complex operator+(Complex n1,Complex n2);
};
//    c1(rel:5,img:3)->n1, c2(rel:3,img:7)->n2
Complex operator+(Complex n1,Complex n2)
{
    Complex ans;//new ans (0,0)
    ans.rel=n1.rel+n2.rel;//ans(rel<<<----n1.rel+n2.rel)
    ans.img=n1.img+n2.img;//ans(img<<<----n1.rel+n2.rel)
    //ans(rel:8,img:10)
    return ans;// ans(rel:8,img:10)--->main
}
int main()
{
    Complex c1;
    c1.set_amount(5,3);
    Complex c2;
    c2.set_amount(3,7);
    c1.display();
    c2.display();
    Complex total=c1+c2; // total=ans(rel:8,img:10)<----complex operator+( c1(rel:5,img:3)
c2(rel:3,img:7))
    //total(rel:8,img:10)<----ans(rel:8,img:10)
    total.display();
    return 0;
}

```

```

#include <iostream>
using namespace std;

```

```

class Complex
{
private:
    int img,rel;

public:
    void set_amount(int rel,int img)
    {
        this->rel=rel;
        this->img=img;
    }
}

```

```

    }
    void display()
    {
        cout<<"\n"<<rel<<" + "<<img<<" i";
    }
    Complex operator-(Complex n2)//c1 is calling operator-, c2 is passed->n2 #member
function
    {
        Complex ans;//new ans (0,0)
        ans.rel= rel-n2.rel;
        //new created c1 c2->n2
        ans.img=img-n2.img;
        return ans;// ans(rel:8,img:10)--->main
    }
    friend Complex operator+(Complex n1,Complex n2);
};
// c1(rel:5,img:3)->n1, c2(rel:3,img:7)->n2
Complex operator+(Complex n1,Complex n2)
{
    Complex ans;//new ans (0,0)
    ans.rel=n1.rel+n2.rel;//ans(rel<<<----n1.rel+n2.rel)
    ans.img=n1.img+n2.img;//ans(img<<<----n1.rel+n2.rel)
    //ans(rel:8,img:10)
    return ans;// ans(rel:8,img:10)--->main
}
int main()
{
    Complex c1;
    c1.set_amount(5,3);
    Complex c2;
    c2.set_amount(3,7);
    c1.display();
    c2.display();
    Complex total=c1+c2; // total=ans(rel:8,img:10)<-----complex operator+( c1(rel:5,img:3)
c2(rel:3,img:7))

    total.display();
    Complex difference=c1-c2; // total=ans(rel:8,img:10)<-----complex operator+(
c1(rel:5,img:3) c2(rel:3,img:7))

    difference.display();
    return 0;
}

```

Operator Overloading in C++ — Simplest Explanation

Let's understand it with **one simple example**.

1. What is Operator Overloading?

Suppose we have two objects:

Number n1, n2;

Normally, C++ knows how to do:

10 + 20

But it does **not automatically know** what this means:

n1 + n2

So, we can **teach C++ what + should do for our objects**.

This is called **operator overloading**.

Operator overloading = giving an existing operator a meaning for objects.

2. Simplest Example

Let's create a **Number** class.

```
#include <iostream>
using namespace std;
```

```
class Number
{
    int x;
```

```
public:
```

```
    Number(int a)
    {
        x = a;
    }
```

```

Number operator+(Number n)
{
    Number temp(0);

    temp.x = x + n.x;

    return temp;
}

void display()
{
    cout << x;
}

};

int main()
{
    Number n1(10);
    Number n2(20);

    Number n3 = n1 + n2;

    n3.display();

    return 0;
}

```

Output

30

3. Understand This One Line

The most important line is:

```
Number operator+(Number n)
```

This means:

"When **+** is used between two **Number** objects, execute this function."

So when we write:

```
Number n3 = n1 + n2;
```

C++ effectively does:

```
n1.operator+(n2);
```

So:

```
n1 = 10
```

```
n2 = 20
```

```
n1 + n2
```

↓

```
operator+
```

↓

```
10 + 20
```

↓

```
30
```

That's operator overloading. **No rocket science—just teaching an operator how to understand your object.**

4. Member Function Operator Overloading

In the above example, `operator+( )` is **inside the class**.

```
class Number
{
public:

    Number operator+(Number n)
    {
        ...
    }
};
```

Therefore, it is called **member function operator overloading**.

The calling object is:

```
n1
```

and the second object is passed as a parameter:

```
n2
```

Conceptually:

n1 + n2

becomes:

n1.operator+(n2);

Remember:

Member Operator

n1 + n2

↓

n1.operator+(n2)

5. Friend Function Operator Overloading

Now let's do the same thing using a **friend function**.

```
#include <iostream>
```

```
using namespace std;
```

```
class Number
```

```
{
```

```
    int x;
```

```
public:
```

```
    Number(int a)
```

```
{
```

```
    x = a;
```

```
}
```

```
    friend Number operator+(Number n1, Number n2);
```

```
    void display()
```

```
{
```

```
    cout << x;
```

```
}
```

```
};
```

```
Number operator+(Number n1, Number n2)
```

```
{
```

```
    Number temp(0);
```

```
    temp.x = n1.x + n2.x;
```

```
    return temp;
}

int main()
{
    Number n1(10);
    Number n2(20);

    Number n3 = n1 + n2;

    n3.display();

    return 0;
}
```

Output:

30

Here:

```
friend Number operator+(Number n1, Number n2);
```

makes `operator+()` a **friend** of the class.

Therefore, even though the function is **outside the class**, it can access:

```
n1.x
n2.x
```

because `x` is private.

6. Member vs Friend — The Main Difference

This is the most important part.

Member Function

Function is **inside class**

`operator+()` is a member

Calling object is implicit

Friend Function

Function is **outside class**

`operator+()` is a non-member friend

Both objects are parameters

`n1.operator+(n2)`

Usually takes **one parameter** for binary `+`

Can directly access private data

`operator+(n1,n2)`

Takes **two parameters** for binary `+`

Can access private data because it is declared `friend`

1. What is a Static Member?

A **static member** belongs to the **class**, not to individual objects.

Normally, every object gets its own copy of a data member:

```
class Student
{
    int marks;
};
```

If we create:

```
Student s1;
Student s2;
```

then:

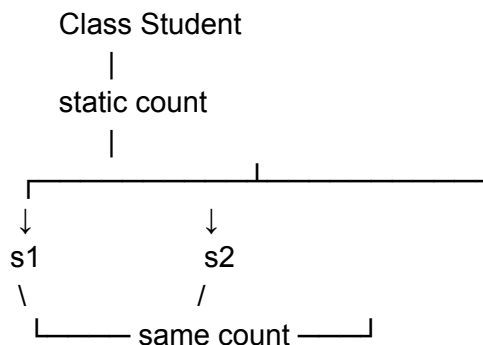
```
s1 → marks
s2 → marks
```

There are **two separate copies**.

But if we declare:

```
static int count;
```

there is only **one copy shared by all objects**.



2. Why Do We Use Static Members?

We use static members when some information should be **common to all objects**.

Common examples:

- Count number of objects
- Store a common company name

- Maintain a common counter
 - Track total students
 - Store common configuration/value
-

3. Simple Example — Count Objects

```
#include <iostream>
using namespace std;

class Student
{
    static int count;

public:

    Student()
    {
        count++;
    }

    void showCount()
    {
        cout << "Number of students: " << count << endl;
    }
};

// Define static data member outside the class
int Student::count = 0;

int main()
{
    Student s1;
    Student s2;
    Student s3;

    s3.showCount();

    return 0;
}
```

Output

Number of students: 3

4. Why Do We Write This?

This line is very important:

```
int Student::count = 0;
```

When we declare:

```
static int count;
```

inside the class, we are declaring the static member.

We then need to **define it** outside the class:

```
int Student::count = 0;
```

Think:

Inside class

↓

Declaration

Outside class

↓

Definition / storage

In modern C++, `inline static` is another option that allows the definition to be placed inside the class.

5. How Does the Counter Work?

Initially:

```
count = 0
```

Create first object:

```
Student s1;
```

Constructor runs:

```
count++;
```

Now:

```
count = 1
```

Create second:

Student s2;

Now:

count = 2

Create third:

Student s3;

Now:

count = 3

All three objects are using the **same count**.

6. Static Member Function

C++ also allows us to create a **static member function**.

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
class Student
```

```
{
```

```
    static int count;
```

```
public:
```

```
    Student()
```

```
    {
```

```
        count++;
```

```
    }
```

```
    static void showCount()
```

```
    {
```

```
        cout << "Number of students: " << count << endl;
```

```
    }
```

```
};
```

```
int Student::count = 0;
```

```
int main()
{
    Student s1;
    Student s2;
    Student s3;

    // Calling static function using class name
    Student::showCount();

    return 0;
}
```

Output:

Number of students: 3

Notice:

Student::showCount();

We don't need an object to call a static member function.

7. Most Important Rule of Static Function

A static member function **cannot directly access non-static data members**.

Example:

```
class Student
{
    int marks;          // Non-static
    static int count;   // Static

public:

    static void display()
    {
        cout << count; // 
        cout << marks; // 
    }
};
```

Why?

Because **marks** belongs to a particular object.

But `count` belongs to the class.

A static function does not have a particular object associated with it.

8. Static Data vs Normal Data

Normal Data Member	Static Data Member
Belongs to object	Belongs to class
Each object has its own copy	One shared copy
Accessed using object	Can be accessed using class
<code>s1.marks</code>	<code>Student::count</code>
Different value for each object	Common value

9. Static Function vs Normal Function

Normal Member Function	Static Member Function
Associated with object	Associated with class
Called using object	Can be called using class
Can access static members	Can access static members
Can access non-static members	Cannot directly access non-static members
Has <code>this</code> pointer	No <code>this</code> pointer

Example:

```
s1.display();
```

Normal function.

```
Student::showCount();
```

```
#include <iostream>
using namespace std;
/*
```

Static membership is a special type of membership where data/function only exists with the class, hence unique and singular.

The most common use of static membership is providing a serial number or sequence number in real programming.

```
*/
```

```
class Student
{
    public:
        static int count;
        int roll_no;
public:

    Student()
    {
        count++;
        roll_no=count;
        cout<<"\nNew student registered with roll no:"<<roll_no;
    }
}
```

static void showCount()//Static function member :Can only access static data, nothing other than that.

```
{
    cout << "Total count till now is: " << count << endl;
    //Within the class we can directly write `count` or `class_name::count`.
}
};
```

// Define static data member outside the class

```
int Student::count;
```

```
//<dt> class::<var>=<initial value>;
```

```
int main()
```

```
{
```

```
    Student s1;
```

```
    s1.showCount();
```

```
    Student s2;
```

```
    s2.showCount();
```

```
    Student s3;
```

Student::showCount();//Even a static member function has to be accessed using scope resolution.

```
    cout<<"\nAccessing count:"<<Student::count;
```

```
public, //When a static member is to be accessed outside the class, provided it is
        //one must specify the name of the class, followed by scope resolution(::),
        //followed by the name of the variable.
    return 0;
}
```